

문제정의서 (연구개발계획서)

4주차 · 문제 정의 확정과 연구개발 계획 수립

프로젝트명	정적분석 기반 소크라틱 페어코딩 웹서비스
조	7 조
지도교수	조은선 교수 (서명)
조원	202203592 심윤진, 202103585 강주형
제출일자	2026. 10. 04
GitHub 저장소	https://github.com/ebs7080-stack/-AI-

Document Revision History

Rev#	Date	Affected Section	Author
1	2026/10/04	최초 작성	홍길동

※ 분량: 본문(1~4장) 5쪽 내외 — 표 · 그림을 포함한 기준이며, 참고문헌과 부록은 분량에 포함하지 않는다.

※ 본 문서는 3주차 「브레인스토밍 결과(양식 3)」에서确定的한 문제 정의문과 최종 아이디어를 출발점으로 작성한다.

1. 연구개발의 배경 및 필요성

1.1 문제 정의 (3주차 결과 확정)

※ 양식 3 §2.1의 최종 문제 정의문을 그대로 옮기고, 이번 주차에 확정된 표현으로 다듬는다.

문제 정의문 (HMW)	어떻게 하면 우리가 정답을 바로 알려주지 않으면서도, 초보 학습자가 스스로 오류 원인을 이해하고 설명할 수 있도록 단계적으로 이끌 수 있을까?
대상 이해당사자	① 주 사용자: 프로그래밍을 처음 배우는 중·고등학생 ② 부 사용자: 다수 학생의 코드를 지도하는 정보 교사
핵심 고충·니즈 (pain / gain)	Pain: 학생은 오류 메시지(영어, 기술 용어)를 이해하지 못해 어디서부터 고쳐야 할지 모릅니다. 교사는 학생 수가 많아 개별 첨삭 시간이 부족하고 같은 설명을 반복합니다. 기존 도구는 통과/실패나 오류 줄 번호만 알려줍니다. Gain: 학생은 정답을 베끼지 않고 스스로 원인을 깨닫는 경험(메타인지)을 얻습니다. 교사는 1차 피드백이 자동화되고 학급 취약점을 한눈에 파악할 수 있습니다.

1.2 국내·외 연구개발 현황과 기존 솔루션의 한계

※ 양식 2 §2.2(기존·유사 솔루션 조사)를 확장한다. 논문·제품·서비스를 구분하고 출처를 반드시 남긴다.

번호	구분 (국내/국외)	사례·논문·제품명	주요 내용	한계점·미충족 니즈	출처
1	국내	백준, 코드업 등 온라인 사이트	문제를 풀고 코드를 제출하면 자동으로 채점	정답/오답과 오류 종류만 표시하며, 왜 틀렸는지에 대한 교육적 설명이 없음	각 서비스 웹사이트
2	국외	Python Tutor	코드 실행 과정을 단계별로 시각화	실행 흐름은 보여주지만 오류 원인을 해석하거나 힌트를 주지 않음	Python Tutor 사이트
3	국외	Pylint 정적분석 도구 등	문법·스타일 오류를 탐지	영어로 된 전문가용 메시지라 초보자가 이해하기 어려움	pylint 공식 문서

번호	구분 (국내/국 외)	사례 · 논문 · 제품명	주요 내용	한계점 · 미충족 니즈	출처
4	국외	ChatGPT 등 생성성 AI	LLM이 대화형으로 코드를 도와줌	범용 LLM은 정답 코드를 바로 줘서 학습 효과가 떨어짐. 또한 학교에서 학생의 성취를 위해 LLM 사용을 제한 하고 있음	

1.3 연구개발의 필요성 및 중요성

※ 위 현황과 한계를 근거로, 이 프로젝트가 왜 필요하고 왜 중요한지를 기술한다.

정보 교과가 확대되며 코딩을 처음 접하는 학생이 늘고 있으나, 입문자는 오류 메시지를 해석하지 못해 곧바로 범용 AI나 검색으로 정답을 얻는다. 이 경우 당장은 해결되지만 원인을 이해하지 못해 같은 오류가 반복된다. 기존 솔루션은 정답을 직접 주거나(범용 LLM), 판정만 하거나, 전문가용 메시지를 출력(정적분석 도구)하여 입문자의 이해를 돕는 데 한계가 있다. 교육적으로 하위권 학습자는 자신의 이해 부족을 스스로 점검하기 어렵고, 현재 수준에 맞춘 scaffolding과 "다음 단계"를 안내하는 형성적 피드백, 적절한 시도를 거치게 하는 생산적 실패, 도움 요청 시점을 돕는 설계가 필요하다. 따라서 정적분석으로 오류 근거를 결정적으로 확보하고 LLM이 이를 바탕으로 질문·단계적 힌트를 주는 에이전트는 환각·정답 노출 위험을 낮추고, 교사의 개별 지도 부담을 줄이며, 학습 이력으로 취약점을 파악하게 한다는 점에서 필요하고 중요하다.

2. 연구개발의 목표 및 내용

2.1 최종 목표

※ "누구의 어떤 니즈·pain을 어떻게 해결한다"가 한 문장에 드러나게 쓴다.

코딩 입문 단계의 중·고등학생이 오류의 원인을 이해하지 못한 채 정답을 복사하는 pain을, Python 정적분석으로 오류 근거를 찾고LLM이 소크라테스식 질문과 단계적 힌트로 안내하는 정적분석 기반 소크라틱 페어코딩 웹서비스로 해결한다.

2.2 컨셉 개발 결과

2.2.1 고정 주제(코어 기능) 선정

※ 고정 주제는 컨셉에 반드시 포함되어야 할 핵심 아이디어(코어 기능)이다. 의사결정 그리드(중요도 × 실현가능성) 또는 멀티보팅 결과를 근거로 적는다.

번호	고정 주제 (코어 기능)	포함된 아이디어 (양식 3 §3.3의 번호)	선정 근거
1	정적분석 기반 오류·스타일 탐지	오류 위치 표시·오류 유형 분류·스타일 점검	피드백의 근거를 결정적으로 확보(환각 방지). 중요도 높음, 실현가능성 높음(Python ast)
2	소크라테스식 교육적 피드백	정답 대신 질문·유도·개념 상기	프로젝트의 핵심 차별점. 중요도 최상, 실현가능성 보통(프롬프트 설계·검증 필요)
3	단계적 힌트 시스템	힌트 1→3단계 상승·정답 비노출	비계·생산적 실패 이론의 구현. 중요도 높음, 실현가능성 높음
4	학습 이력·취약점 통계	오류 이력 저장·반복 오류·교사용 통계	교사 pain 해소와 개인화의 기반. 중요도 중상, 실현가능성 보통(기본 이력부터 구현)

2.2.2 넵킨 피치 (NABC)

N (Need) — 해결해야 할 충족되지 않은 니즈	A (Approach) — 니즈를 충족하기 위한 접근법
코딩 입문 중·고등학생은 오류 메시지를 이해하지 못하고 무엇을 모르는지 설명하지 못해 정답지에 의존한다. 교사는 학생별 즉각 피드백이 어렵다.	Python ast 정적분석으로 오류 위치·유형을 확인한 뒤, LLM이 이를 근거로 정답 대신 질문과 단계적 힌트를 제공한다. 학습 이력을 누적해 교사에게 취약점 통계를 보여준다.
B (Benefit) — 고객이 얻는 혜택 / 우리가 얻는 혜택	C (Competition) — 경쟁 상대와 우리의 강점

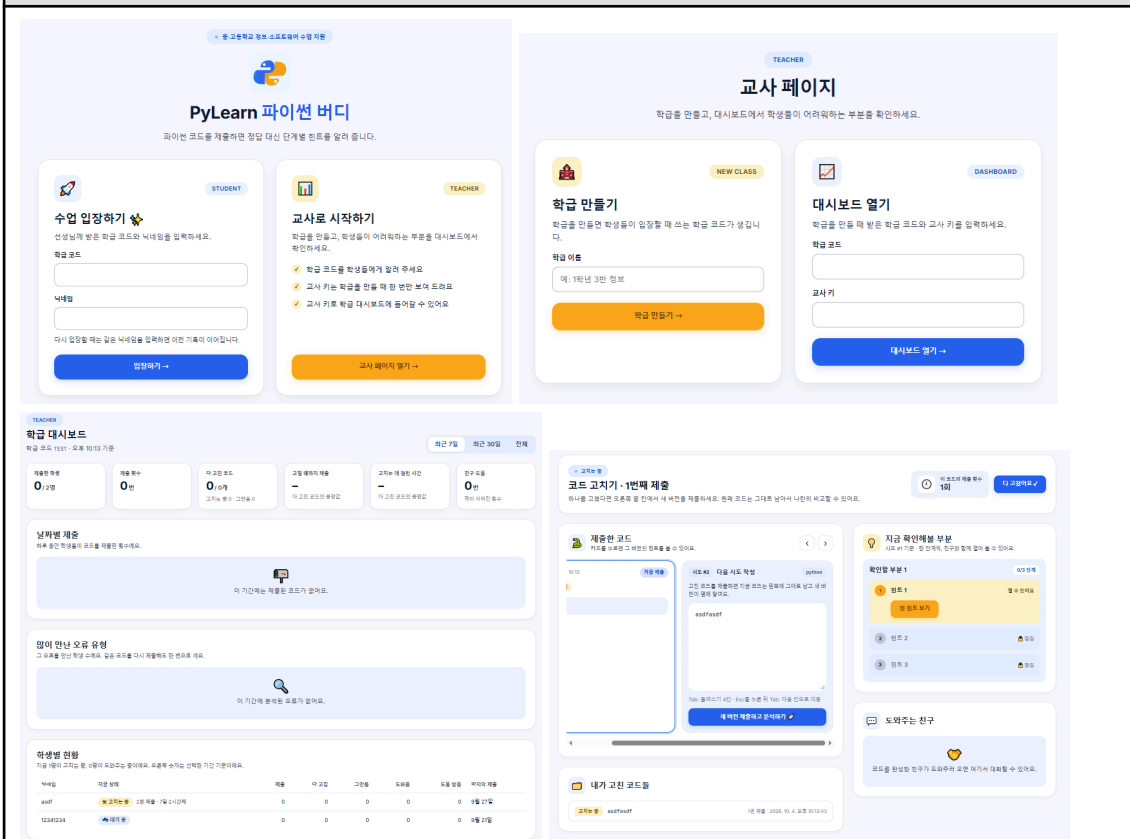
• 고객(학생 및 교사): 스스로 오류를 찾는 경험, 수준에 맞는 힌트, 반복 오류 감소, 교사의 취약점 파악 • 개발자: 교육공학 + AI 융합 결과물, 현장 검증 가능한 최종 보고서(소논문)와 경진대회 성과	• 경쟁: 범용 LLM(정답 직접 제공), 온라인 Judge(판정만) • 강점: 정적분석 근거 + 교육 이론 기반 힌트 단계 + 한국어·국내 정보 교과 맥락 + 교사용 통계
--	---

2.2.3 컨셉 시트

※ 컨셉은 3~4개를 만들어 보고 그중 하나를 확정한다. 확정된 컨셉을 아래에 작성한다.

컨셉명	학생 코딩 자동 피드백 및 오류 통계 정리	슬로건	좋은 수업을 위한 자동 피드백 및 오류 통계
목표 고객	프로그래밍 입문 단계의 중·고등학생과 정보 교사	가치 제안(gain)	스스로 원인을 찾아 고치는 성취 경험, 이해 가능한 한국어 설명, 교사의 피드백 부담 감소, 학생들의 오류 통계화

컨셉 이미지 · 스케치 · 스토리보드 (제품: 이미지/활용 상황, 서비스: 프로세스 맵, 앱: 핵심 화면)



주요 기능 · 스펙 · 특징 (코어 기능과 추가 · 부가 기능을 구분하여 기술)

코어 기능 : 정적분석 오류 분류, 자기설명 입력, 단계적 힌트, 쉬운 오류 설명, 교사·학생 로그인과 학급 관리. 교사용 학생의 오류 분포 대시보드, 부가 기능 : 이미 완료한 학생을 튜터로 매칭 시키는 시스템
유저 인사이트 (컨셉을 이끌어낸 사용자의 통찰 · 니즈 · pain point) 학생은 오류 메시지를 이해하지 못해 어디서부터 봐야 할지 모르고, 자기 코드의 의도를 말로 설명해 본 경험이 없습니다. 수업은 정답 코드 제시 위주라 '왜 그렇게 생각했는지' 되물어 주는 사람이 없습니다. 교사는 같은 오류에 같은 설명을 매번 다시 씁니다

2.3 개발 범위

※ 컨셉 시트의 기능을 이번 학기에 개발할 것과 이후로 미룰 것으로 나눈다. 여기서 정리한 코어 기능이 6주차 Product Backlog의 입력이 된다.

번호	구분 (코어/부가/ 제외)	기능명	설명	판단 근거
1	코어	정적 분석 기반 오류 분류	제출 코드(Python)를 분석해 오류 유형을 분류	모든 피드백의 입력이 되는 기반 기능
2	코어	단계적 힌트	분석된 오류 기반 단계적 힌트 피드백	소크라테스식 교육적 피드백
3	코어	교사용 학생 오류 대시보드	반복 오류 분포 시각화	교사의 수업 발전을 위한 자료
4	부가	학습 이력 저장 · 기본 통계	제출·오류 유형 이력 저장, 기본 취약점 통계 화면	교사 pain 해소. 이번 학기는 기본 수준만
5	제외	적응형 난이도 · Java 지원	수준별 난이도 조정, 다중 언어	이번 학기 범위 초과. 2학기(27-1) 확장

2.4 달성 지표

※ 1주차 디자인 개요서에서 잠정치로 적었던 지표를 여기서 확정한다.

번호	구분 (정량/정성)	지표	목표치	측정 방법 · 시점
1	정량	정적분석 오류 탐지율	입문자 오류 코드 샘플 50개 중80% 이상	정답 라벨과 비교 / Sprint 1 Review(11월 중순)
2	정량	정답 비노출 준수율	95% 이상 (정답 코드가 직접 포함된 응답 5% 이하)	테스트 입력 30건 응답 점검 / Sprint 2 Review

번호	구분 (정량/정성)	지표	목표치	측정 방법 · 시점
3	정량	피드백 응답 시간	평균 10초 이내	서버 로그 측정 / Sprint 1~2
4	정성	학습자 유용성·이해도	5점 척도 평균 4.0 이상 + 인터뷰	입문 학습자 5명 이상 설문·인터뷰 / Sprint 2 Review(12월)

3. 연구개발의 추진전략 및 방법

3.1 AS-IS 시나리오 (현재 사용자의 경험)

※ 사용자가 현재 어떤 순서로 행동하는지를 번호를 붙여 기술하고, 마지막에 문제점을 명시한다.
가능한 한 여러 개를 만들어 본다.

[시나리오 1 — 학생]

1. 파이썬 실습 과제 코드를 작성·실행한다. 2. 영어 트레이스백(IndentationError, NameError 등)이 출력된다. 3. 영어로 된 에러메시지를 이해하기 어려워 어디가 왜 문제인지 알 수 없다. 4. 질문할 말을 찾지 못해 인터넷에 검색하거나 범용 AI에 코드를 붙여넣는다. 5. 완성된 코드를 그대로 복사해 수업시간 실습에 통과한다. 6. 이유를 모른 채 다음 과제에서 같은 오류를 반복한다.

→ 문제점: P1 오류 메시지 해석 곤란 / P2 모르는 것을 설명하지 못해 질문이 어려움 / P3 정답 복사로 사고가 생략되고 오류가 반복됨

[시나리오 2 — 교사]

1. 학급 단위(20~30명)로 실습을 진행한다. 2. 여러 학생이 동시에 도움을 요청한다. 3. 교사가 한 명씩 확인하는 동안 나머지 학생은 막힌 채 대기한다. 4. 수업 후 어떤 개념에서 많이 틀렸는지 확인할 자료가 없다.

→ 문제점: P4 개별 피드백 시간 부족 / P5 학습 취약점 데이터 부재

3.2 TO-BE 시나리오 (디자인 시나리오)

※ 확정한 컨셉을 적용했을 때 사용자의 경험이 어떻게 바뀌는지를 같은 형식으로 기술한다.
AS-IS의 어느 문제점이 해소되는지 대응이 보이게 쓴다.

[시나리오 1' — 학생]

1. 코드를 작성해 정적분석 기반 소크라틱 페어코딩 웹서비스에 제출한다. 2. 정적분석이 오류를 쉬운 한국어로 표시한다 [P1]. 3. 정답 대신 "이 변수가 먼저 선언되었는지 확인해 볼까요?" 같은 질문을 받는다 [P2-P3]. 4. 막히면 힌트를 1→3단계로 받으며 정답 코드는 직접 제공되지 않는다 [P3]. 5. 스스로 수정·재제출하고 원인을 한 줄로 정리한다. 6. 이력이 저장되어 같은 유형이 반복되면 이전 경험을 상기시킨다 [P3].

[시나리오 2' — 교사]

1. 학생들이 각자 정적분석 기반 소크라틱 페어코딩 웹서비스로 1차 피드백을 받는다 [P4]. 2. 교사는 도움이 더 필요한 학생을 중심으로 지도한다. 3. 수업 후 대시보드에서 오류 유형별 통계와 반복 오류 학생을 확인한다 [P5]. 4. 확인한 취약점을 다음 수업에 반영한다.

3.3 프로토타이핑 계획

※ 아이디어 · 컨셉을 가장 간단 · 빠르게 구성해 피드백을 받는 것이 목적이다. 작고 단순하게 시작한다.

번호	프로토타입 형태	목적 (무엇을 확인)	대상 사용자	방법 · 도구	시기
1	실행 가능한 웹 MVP	핵심 파이프라인(탐지→ 질문→힌트) 중단 사용성	코딩 입문 학습자 5명 이상	FastAPI + 웹 화면	9~11주차 (11/2~11/16)

번호	프로토타입 형태	목적 (무엇을 확인)	대상 사용자	방법 · 도구	시기
2	컨셉 시트 · 스토리보드	문제·컨셉이 공감되는지, AS-IS/TO-BE 흐름 이해 여부	팀 내부, 정보 교사·교육 전공 동료	종이 · Notion	6주차 (10/12)
3					
4					

※ 프로토타입 형태 예: 컨셉 시트 · 스토리보드 · 종이(페이퍼) 프로토타입 · UI 목업(Figma) · 유즈케이스 · 유즈 시나리오 · 비디오 프로토타이핑

3.4 테스트 계획

※ 테스트는 디자인 시나리오가 AS-IS를 실제로 개선했는지, 사용자의 니즈 · pain · gain을 반영했는지, 새로운 불편이 생기지 않았는지를 검증하는 과정이다.

단계	계획 내용	시기
Pre-test	대표 오류 코드 5종을 준비하고, 단계적 힌트가 적절히 주어지는지 확인합니다. 어디에서 / 누구에게 / 무엇을 / 어떤 방법으로 테스트할 것인가?	6~7주차
Test	실제 현직 정보 교사에게 웹사이트를 공유 하여 학생들과 실제로 사용해 보도록 하고, 사용 후기및 개선점, 추가로 필요한 기능들을 물어 봅니다. 직접 경험하게 하기 / 관찰하기 / 물어보기 (공감 과정과 동일한 방법)	10주차
Post-test	반응을 정리해 힌트 문구와 단계 수, 화면 흐름을 수정합니다. 수혜자 반응 살펴보기 → 우리의 대안(컨셉 · 프로토타입) 업데이트	11주차

※ 프로토타이핑 → 테스트 → 수정을 반복하고, 그 과정을 반드시 기록으로 남긴다. (무엇이 문제였고 어떻게 개선했는지)

3.5 개발 환경 및 기술 스택

구분	선정 내용	선정 이유
언어 · 프레임워크	백엔드: python 3.12.8 프론트엔드 : HTML CSS JavaScript 프레임 워크 : FastAPI	분석 대상이 Python이라 생태계가 일관되기 때문
데이터 · DB	SQLite	학습 이력 및 오류 통계 저장
하드웨어 · 장비	개인 노트북	별도 장비 필요 없음
협업 · 형상관리	GitHub	수업 규정 및, 공유 편의성

구분	선정 내용	선정 이유
기타(배포 · 테스트)		

4. 연구개발 팀의 구성 및 프로젝트 추진 일정

4.1 팀 구성 및 역할

이름	학번	역할	담당 기능 · 업무	보완이 필요한 역량과 학습 계획
심윤진	20220359 2	팀장 · Product Owner	요구사항·DB 설계, 피드백·힌트 설계(프론트), 문서	웹 프론트엔드·배포 → Sprint 0에서 FastAPI/React 학습
강주형	20210358 5	Scrum Master · 개발	Python ast 정적분석 모듈, 백엔드 API	교육학 기반 피드백 설계·프론트 평가 → 이론 자료 공동 학습

※ 역할 예: 팀장 / Product Owner / Scrum Master / 개발(프론트 · 백엔드 · AI) / 테스트 · 문서

4.2 추진 일정

※ 수업 진행은 강의계획서 기준이며, 팀의 추진 내용과 산출물을 채워 넣는다.

주차	기간	수업 진행 (강의계획서)	팀 추진 내용 · 산출물
5	10/5	Agile(Scrum) 프로세스 소개, 프로토타이핑과 테스트	문제정의서 확정·제출 / 산출물: 문제정의서
6	10/12	Product Backlog 정의 (User Story, 우선순위)	Product Backlog(학생·교사 User Story), 코어 3기능 우선순위 / 산출물: Backlog
7	10/19	Sprint 0 — 시스템 구조 초안 설계, 개발 환경 구축	Sprint 0: 구조(정적분석→DB→웹) 초안, 개발환경·GitHub 구성 / 산출물: 구조 설계서
8	10/26	Sprint 1 시작 — 핵심 기능 1개 선정 및 구현 시작	Sprint 1 시작: 정적분석 오류 탐지(핵심 기능 1개) 구현 시작 / 산출물: 탐지 모듈 초안
9	11/2	Sprint 1 구현 — 실행 가능한 시스템 확보	Sprint 1 구현: 탐지 + 질문 생성 연동, 실행 가능한 파이프라인 / 산출물: 동작 파이프라인
10 ~ 11	11/9 ~ 11/16	Sprint 1 Review — 핵심 기능 실행 검증, 시연	Sprint 1 Review: 탐지를 측정·시연, 프로토타입 테스트·수정 / 산출물: 테스트 기록
12	11/23	Sprint 2 시작 — 기능 확장 및 예외처리 대상 선정	Sprint 2 시작: 힌트·예외처리(빈 코드·구문 오류) 대상 선정, 웹 UI 설계 / 산출물: Sprint 2 백로그
13	11/30	Sprint 2 구현	Sprint 2 구현: 힌트 시스템, 웹 UI, 학습 이력 저장 / 산출물: 웹 MVP

주차	기간	수업 진행 (강의계획서)	팀 추진 내용 · 산출물
14 ~ 15	12/7 ~ 12/14	Sprint 2 Review, 최종 발표 · 시연	Sprint 2 Review, 최종 발표·시연: 지표 측정 / 산출물: 시연, 최종 보고서

4.3 위험 요소 및 대응 방안

번호	예상 위험 요소	영향	대응 방안
1	LLM이 정답 코드를 직접 노출하거나 환각을 일으킴	높음 (학습 효과·신뢰 저하)	정적분석 결과를 근거로 프롬프트 제한, 정답 노출 필터와 검증 테스트(지표 2)
2	정적분석으로 잡지 못하는 논리 오류	중	입문자 빈출 오류부터 규칙화하고 테스트케이스 실행 결과를 보조로 사용, 범위 밖은 LLM 일반 힌트로 처리
3	파일럿 사용자 섭외 및 미성년자 개인정보·동의 문제	중	교사·지인을 통해 소규모 섭외, 익명화·동의 확보·로그 최소 수집, 어려우면 입문 대학생으로 대체
4	소규모 팀·복수전공·타 과목 병행으로 일정 지연	중	Sprint 범위를 코어 3기능으로 제한, 주간 점검, 부가 기능은 다음 Sprint로 이월

5. 참고문헌 (References)

※ 문제정의서 작성에 인용한 논문 · 보고서 · 기사 · 제품 자료를 모두 기재한다. 분량에 포함되지 않는다.

※ 표기 예: [1] 홍길동, 김철수, "논문 제목", 한국정보과학회 논문지, 제52권 제3호, pp.101-110, 2025. [2] 기관명, "보고서 제목", 발행처, 2025. (URL, 접속일)

<황혜민, 이장원. (2025). AI 기반 짝 프로그래밍 기법을 활용한 프로그래밍 수업모델 설계. 컴퓨터교육학회 논문지, 28(1), 1-12. >

6. 제출 전 자가점검 체크리스트

자가점검 항목	확인
프로젝트명 · 조 · 조원 · 지도교수 · 제출일자 · GitHub 저장소 주소를 모두 기재했는가?	☒
§1.1의 문제 정의문이 3주차(양식 3)에서 확정한 문장과 일치하는가?	☒
국내 · 외 현황 조사에 출처를 모두 남겼는가?	☒
최종 목표가 "누구의 어떤 니즈를 어떻게 해결한다"로 읽히는가?	☒
고정 주제(코어 기능)를 선정 근거와 함께 밝혔는가?	☒
넵킨 피치(NABC) 네 칸이 모두 채워졌는가?	☒
컨셉 시트에 컨셉명 · 슬로건 · 목표 고객 · 가치 제안 · 주요 기능 · 유저 인사이트가 모두 있는가?	☒
개발 범위를 코어 / 부가 / 이번 학기 제외로 구분했는가?	☒
달성 지표에 목표치와 측정 방법이 있는가? (1주차의 잠정치로 확정했는가?)	☒
AS-IS와 TO-BE 시나리오가 서로 대응되며, 어느 문제점이 해소되는지 보이는가?	☒
프로토타이핑 계획에 형태 · 목적 · 대상 사용자 · 시기가 있는가?	☒
테스트 계획이 Pre-test / Test / Post-test로 구성되어 있는가?	☒
팀 역할과 주차별 추진 일정이 강의계획서 일정과 맞는가?	☒
본문(1~4장)이 5쪽 내외인가?	☒
[부록] 작성 가이드 페이지를 삭제했는가?	☒
문서를 GitHub 저장소에 업로드했는가?	☒